# Swarm Intelligence

A critical analysis of the main techniques

*Course*: Computational Models for Complex Systems

*Teacher*: Prof. Paolo Milazzo

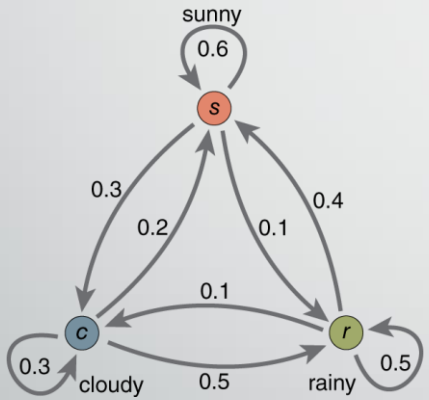
A.A. 2018/2019
VALERIO DE CARO

# Introduction

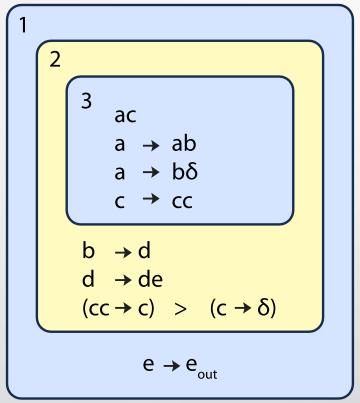
Paving the way for the Swarm Intelligence metaheuristics

# On the modelling of a complex system

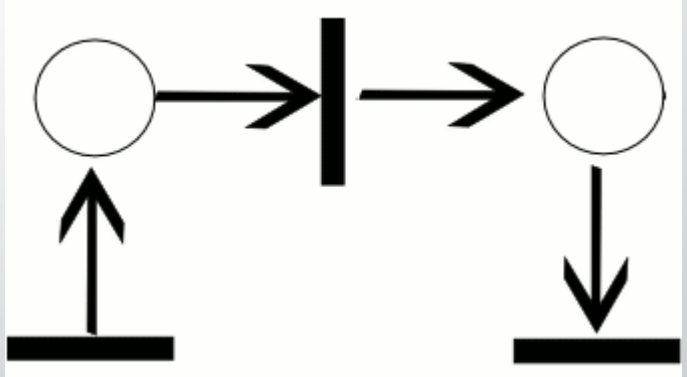
- A **model** is a simplified and approximate representation of a system, that allows reasoning on the system's properties.
- We have approached modelling in several different ways:




**Stochastic Transition Systems**




**P-Systems (Membrane computing)**

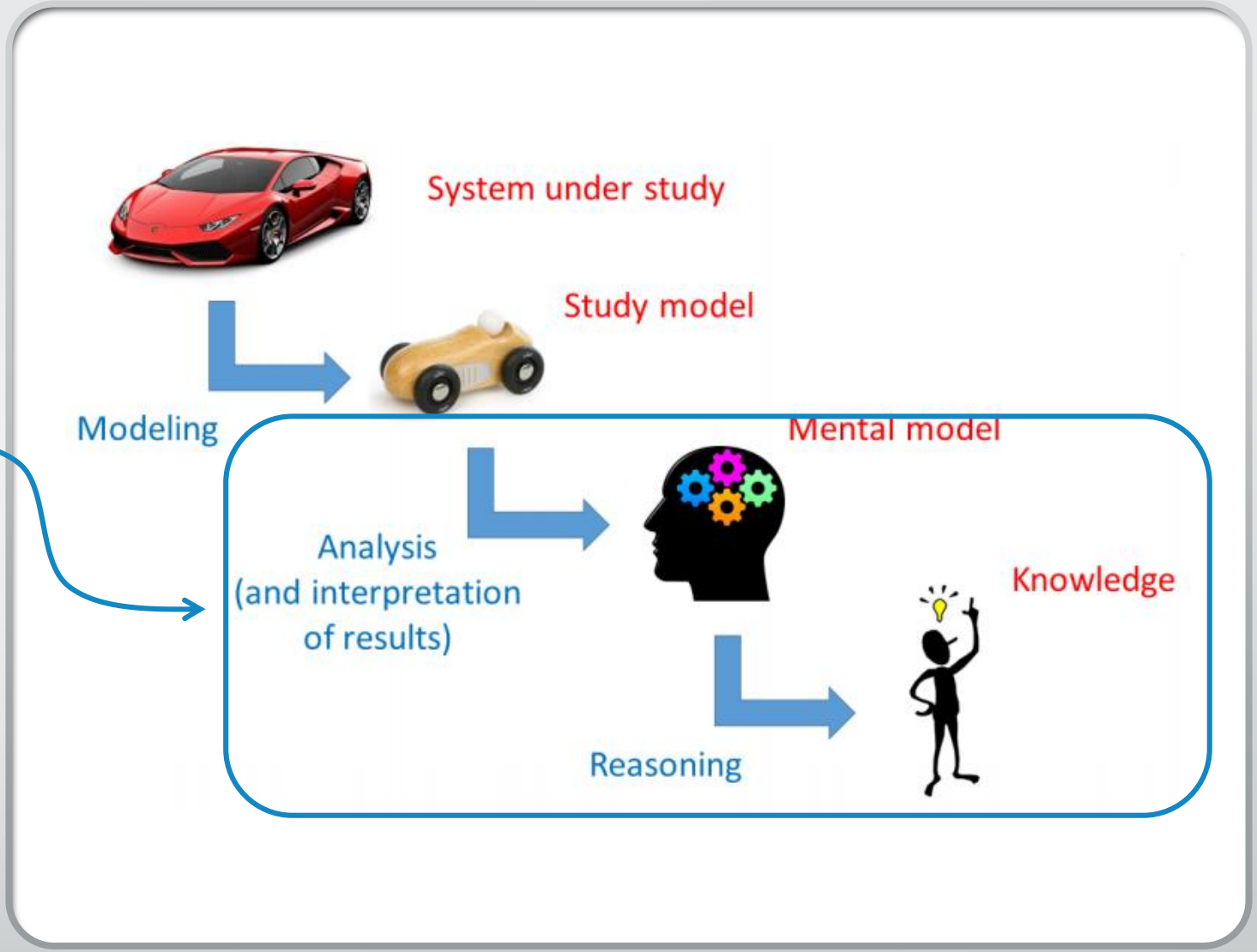


**Petri nets**

and more…

# Why modelling a system?

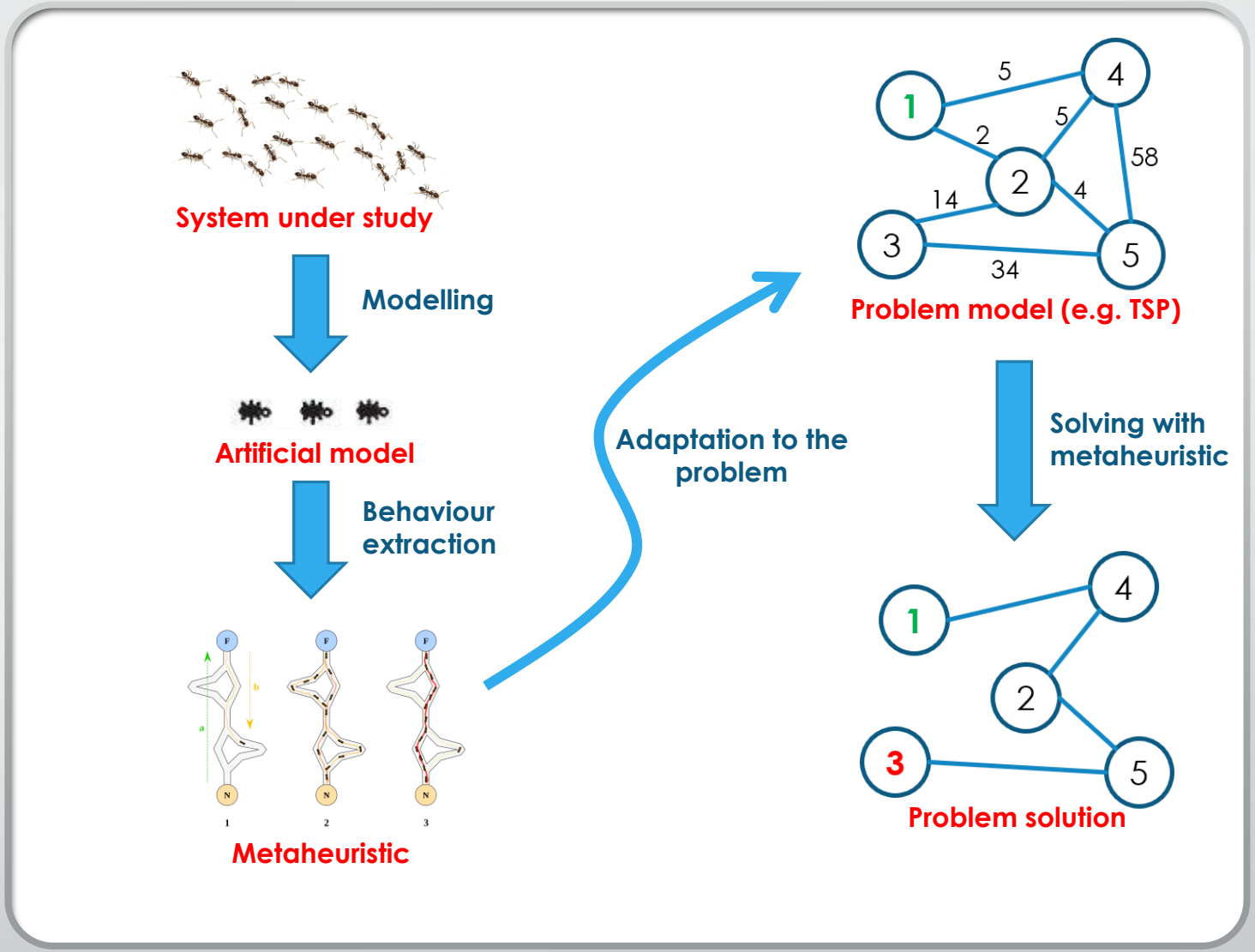
- For a better comprehension of its behaviour;
- **Target:** find a consistent and good representation of the system
- **… what if we change our aim?**




From the slides of the course

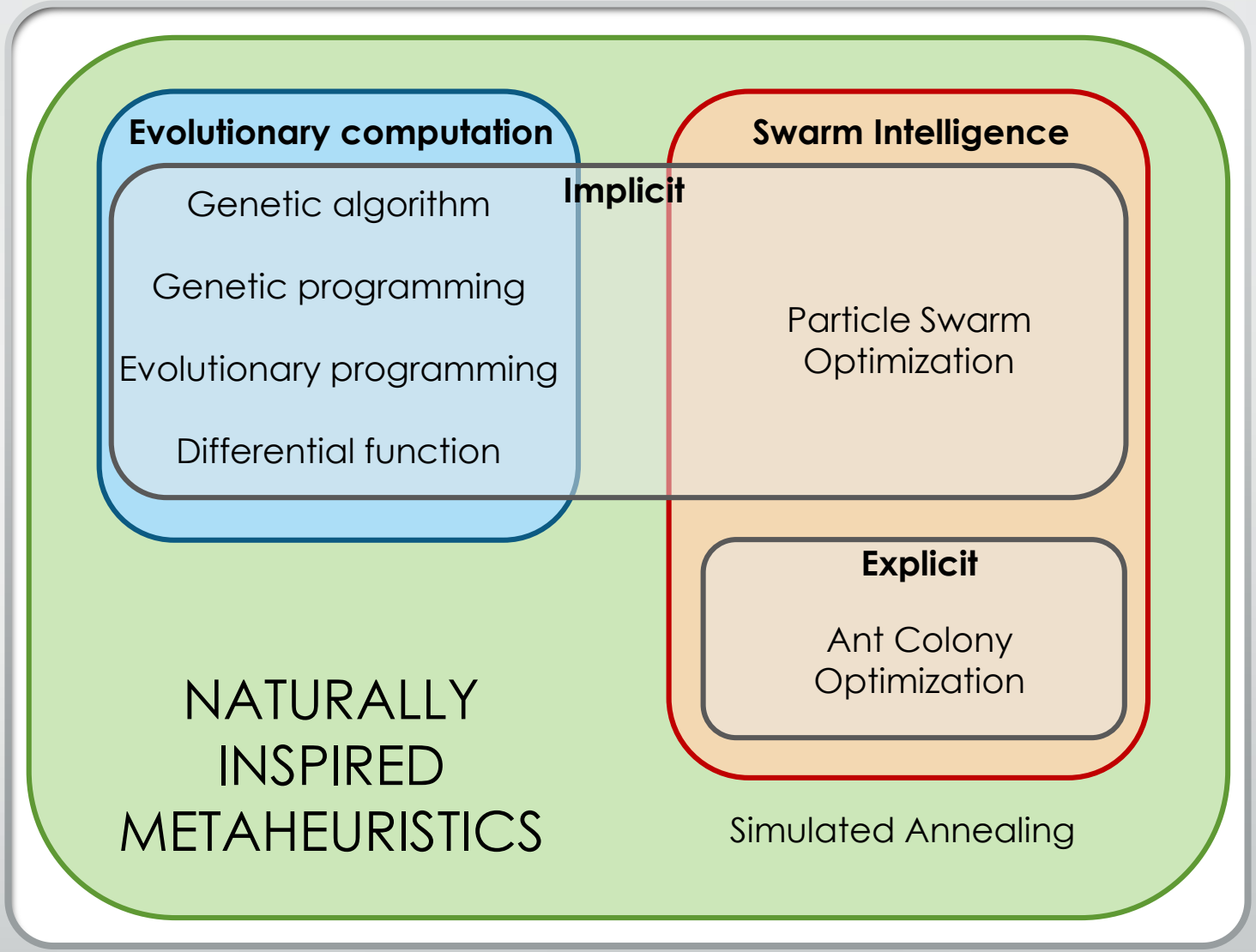
# Optimization with naturally inspired methods

1. Modelling of the natural system;
2. Extraction of a **metaheuristic** (→ a higher-level heuristic designed to find, generate, or select a heuristic);
3. Adaptation of the metaheuristic to a certain problem;
4. Solving the problem with the exploitation of the modelled metaheuristic.



# Optimization with naturally inspired methods

- This approach has given birth to several **naturally inspired metaheuristics**;
- **Evolutionary computation**: a set of metaheuristics based on the evolutionary biology concepts (e.g. natural selection, mutation and fitness).

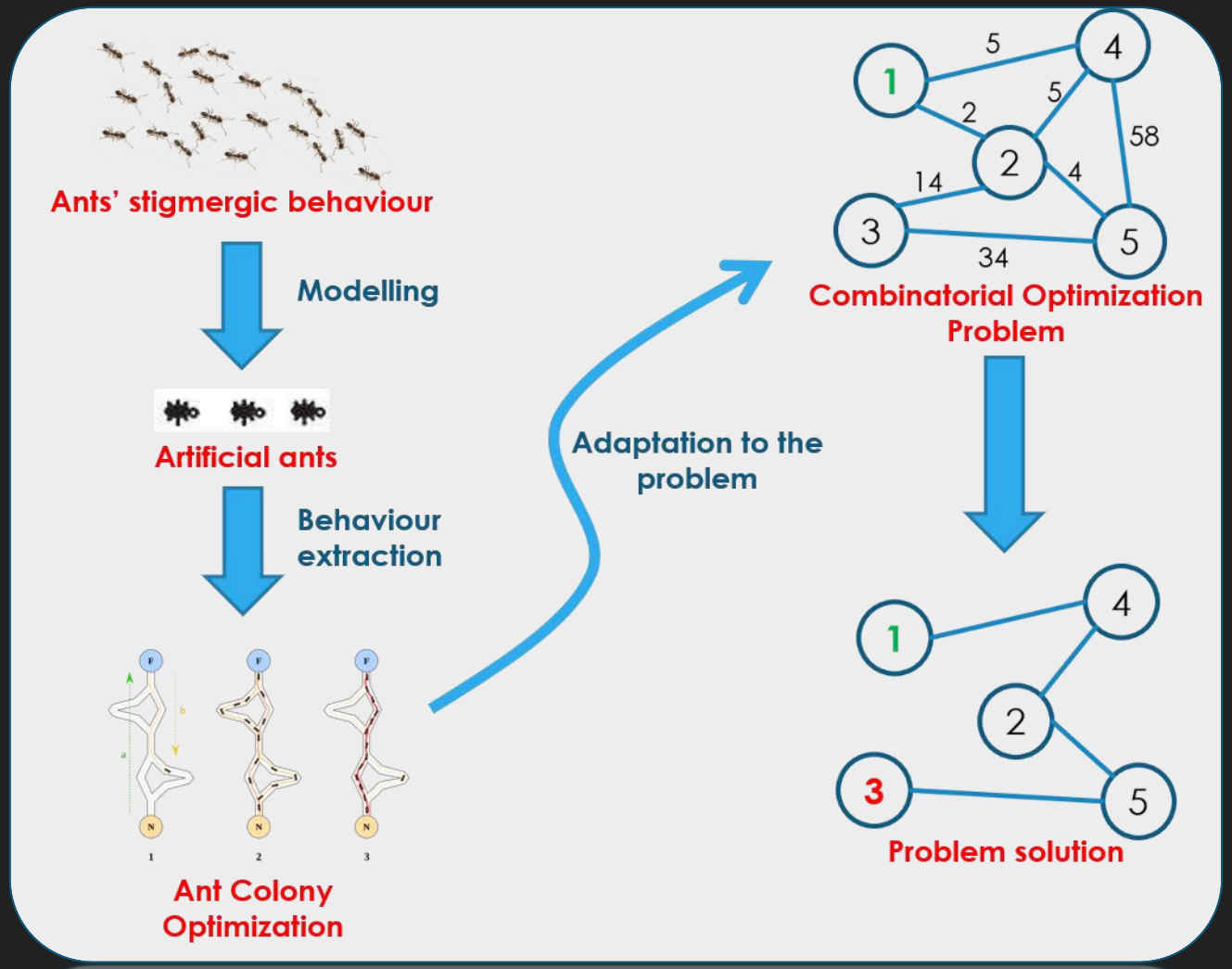


# Swarm Intelligence: the essentials

- Term coined by G. Beni in 1989, it indicates the the **collective behaviour** of **decentralized**, **self-organized systems**, either natural or artificial;
- It has been inspired by the behaviour of **social insects** and **animals:**
  - Agents follow simple rules, and although there is no centralized control structure dictating how they should behave, a **collective intelligent behaviour**, unknown to the individual agents, emerges from their interactions.
- We are going to discuss both Ant Colony Optimization and Particle Swarm Optimization metaheuristics.

# Ant Colony Optimization

**Proposed by M. Dorigo et al. in 1990**



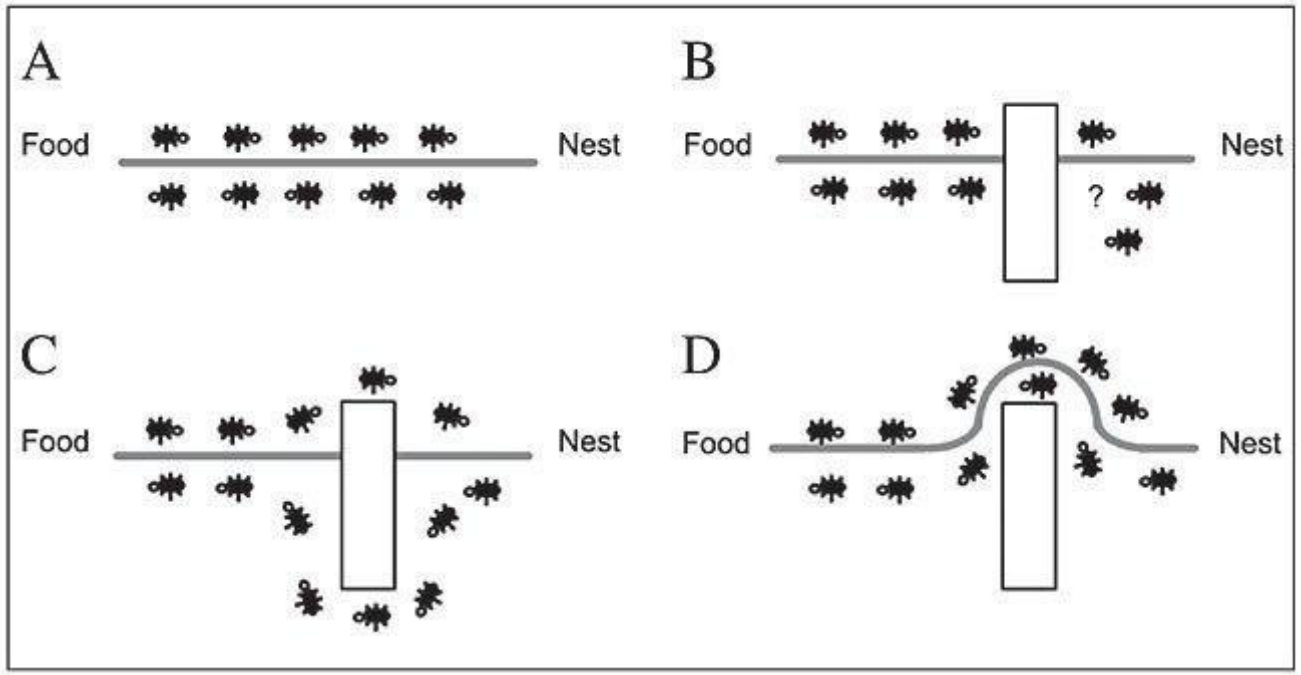
# Ants' stigmergic behaviour

**System under study**

# Ants in nature

- Three facts about ants:
  1. The first species of ants appeared tens of millions of years ago;
  2. They do not have any kind of rational capability. Instead, they communicate only via chemical substances, auditive and tactile signals;
  3. They are blind, but they have impressive capabilities in finding their way and finding food supplies;
- Ants as single individuals seem really weak, so what is the secret of their survival during all this time?
  4. They are, along with bees, one of the best examples of **social insects** that we can find in nature.

# Ants' stigmergic behaviour




- **Situation**: an ant colony follows the path for a located spot of food supplies
  - A. The path is free;
  - B. An obstacle is placed into the path;
  - C. Ants spread in two different paths;
  - D. All the ants converge in the shorter path.
- The change of behavior C→D is possible due to the **stigmergy**:
  - A mechanism of indirect coordination, through the environment, between agents.

# Ants' stigmergic behaviour

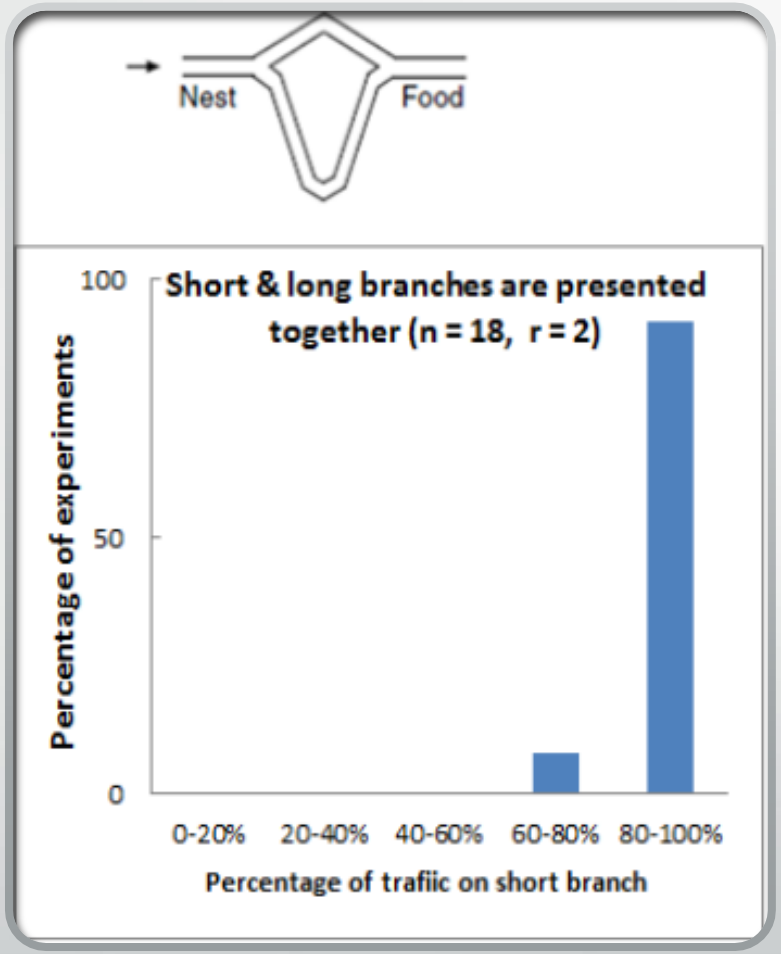
- Stigmergy works thanks to the following conditions:
  1. When ants follow a **path for a food spot** and to return to the nest, **they release food trail pheromone**;
  2. Ants are more likely to select the path with **higher pheromone concentration**;
  3. The **path gets reinforced** every time an ant releases pheromone in it;

- **Idea**: if the path is **shorter**, then the pheromone is **less likely to evaporate** → the concentration is higher and the ants are going to select it.

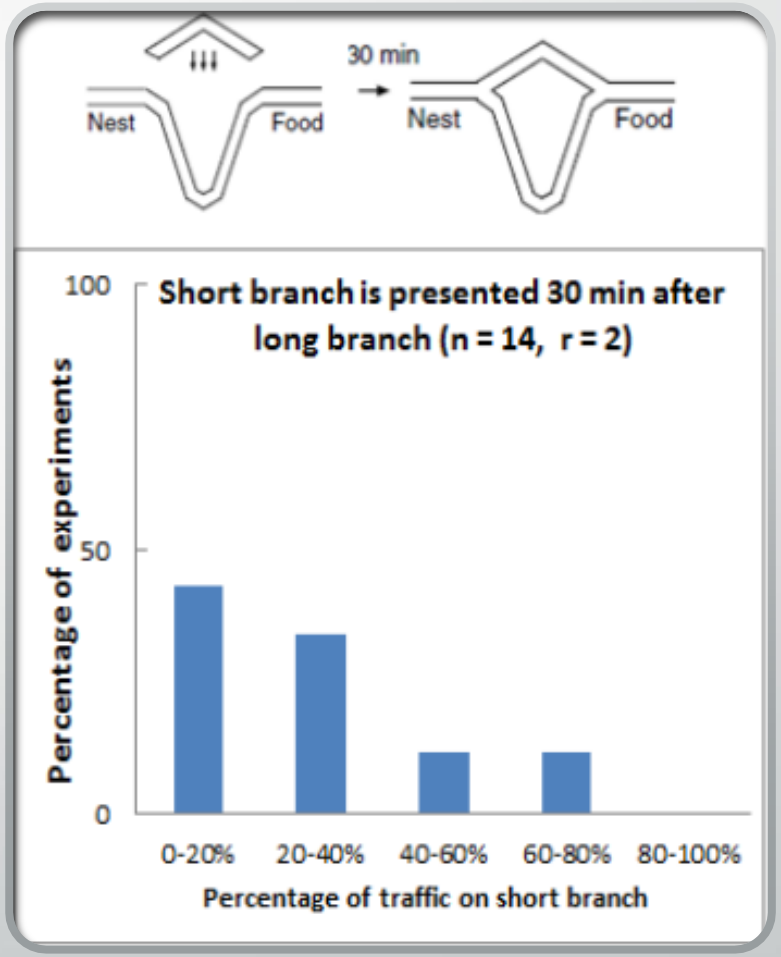
# Double bridge experiment

- This experiment was designed and run by Goss, Deneubourg et al., and evidenced the **failure** of the stigmergic behaviour **through changes in the environment**.

1. Both bridges are available

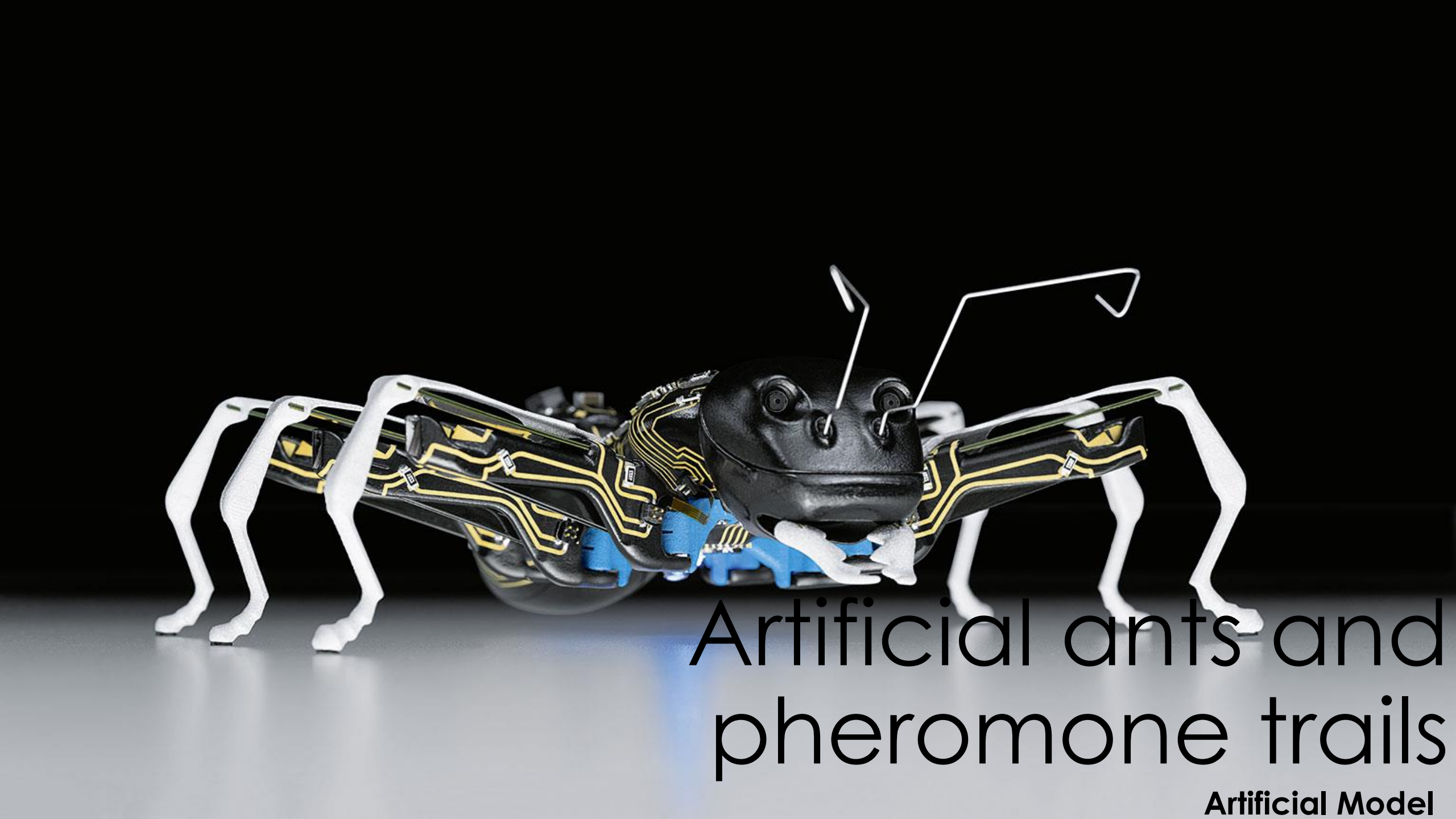



2. Shorter bridge is added 30 minutes later



# Double bridge experiment: our viewpoint

- While modelling the metaheuristic, we must take care of the following take home messages from the experiment:
  - This behaviour does not react well to changes on the system;
  - The stability of a path is provided by the pheromone concentration → its **evaporation rate** is important for it;
  - Ants **often explore new paths**.



# Artificial ants and pheromone trails

**Artificial Model**

| CRITERION | REAL ANTS | ARTIFICIAL ANTS |
|---|---|---|
| **Pheromone depositing behaviour** | Pheromone is deposited **both ways** while ants are moving | Pheromone is often deposited **only on the return way after a candidate solution is constructed and evaluated.** |
| **Pheromone Updating Amount** | The pheromone trail on a path is updated with a pheromone amount that **depends on the quantity and quality of the food** | Once an ant has constructed a path, the pheromone trail of that path **is updated** on its return way **with an amount that is inversely proportional to the path length stored in its memory**. |
| **Memory Capabilities** | Real ants have **no memory** capabilities. | Artificial ants **store the paths they walked** onto in their memory to be used in retracing the return path. They also use its length in determining the quantity of pheromone to deposit on their return way. |
| **Return Path Mechanism** | Real ants **use the pheromone deposited on their forward path** to retrace their return way when they head back to their nest | Since no pheromone is deposited on the forward path, artificial ants **use the stored paths** from their memory to retrace their return way. |

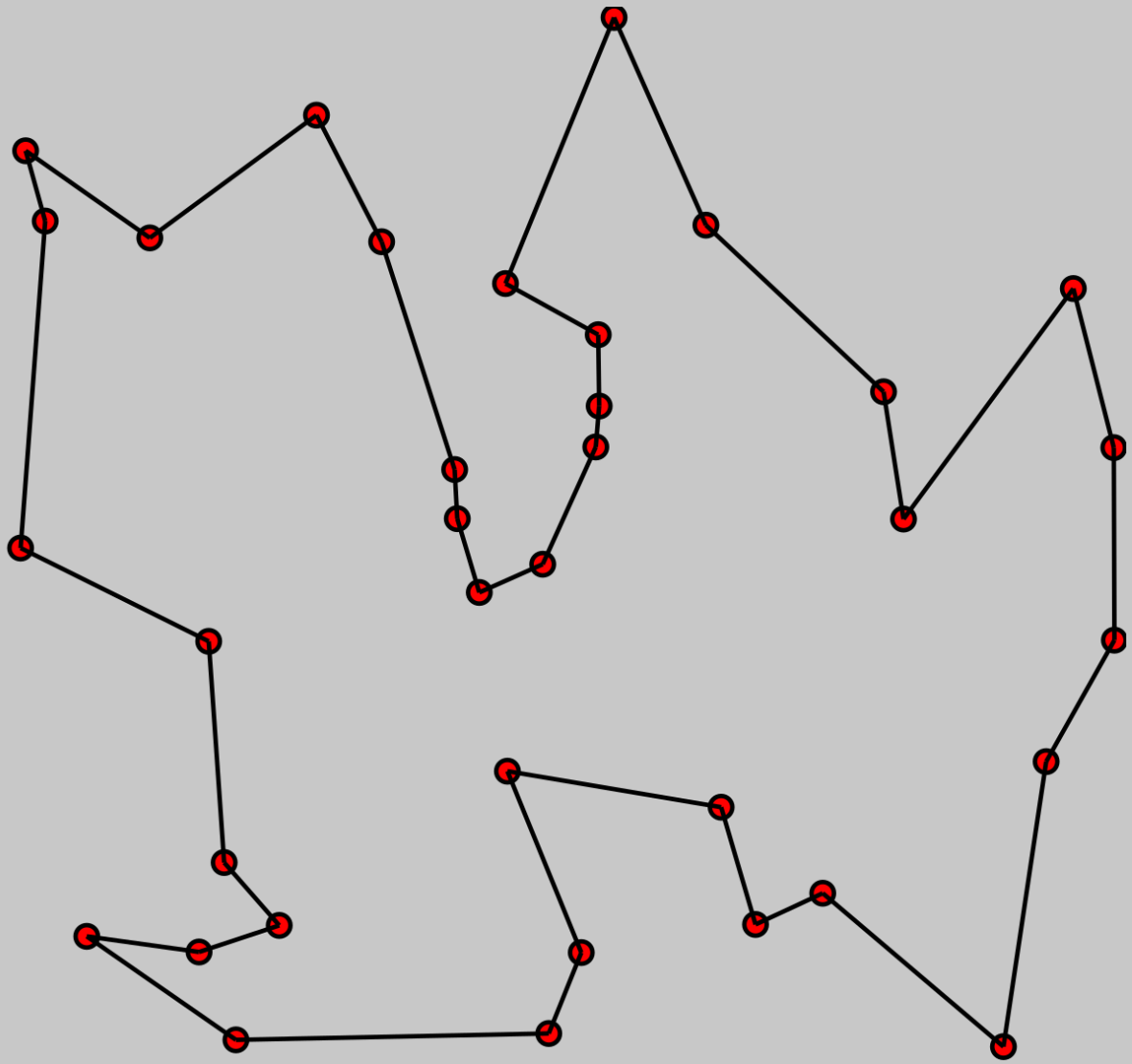
# Modelling the artificial ants

The behaviour of the artificial ants is modeled in order to better fit the definition of the heuristic.

| CRITERION | REAL ENVIRONMENT | HEURISTIC DYNAMICS |
|---|---|---|
| **Pheromone Evaporation Behaviour** | Pheromone **evaporates too slowly** making it less significant for the convergence. | Pheromone **evaporates exponentially** making it more significant for the convergence. |
| **Ecological Constraints** | **Exist**, such as predation or competition with other colonies and the colony's level of protection. | Ecological constraints do not exist in the artificial/virtual world. |

# Modelling the dynamics of the heuristic

Simplification of the environmental characteristics in order to make the heuristic more viable for the search of the solution.

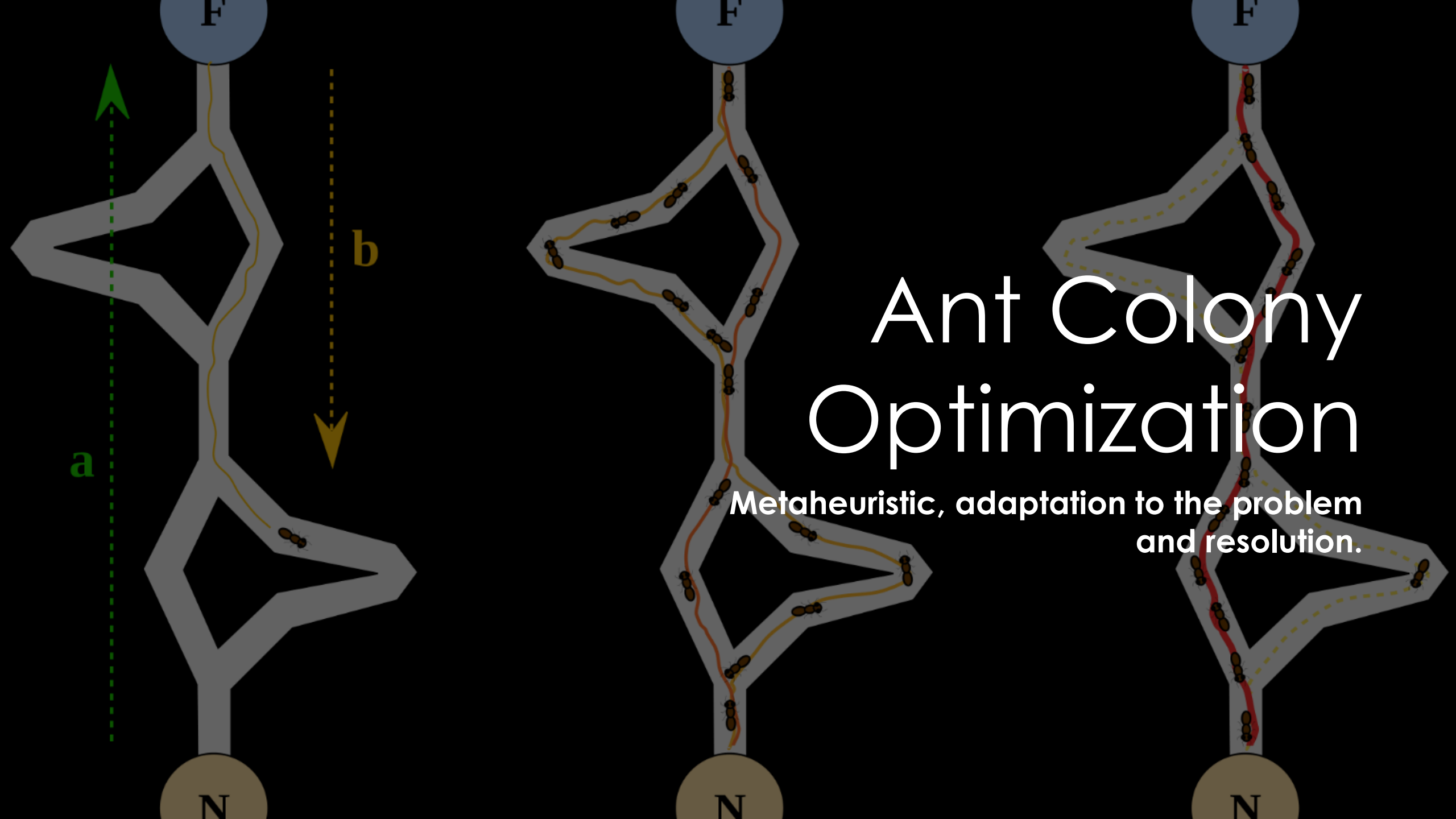


# Combinatorial Optimization Problems

**Problem Model**

# Combinatorial Optimization Problems: definition

- A **model** $P = (S, \Omega, f)$ of a *combinatorial optimization problem* consists of
  - A ***search space*** $S$ that is defined by a finite set of decision variables, each with a finite domain;
  - A ***set of constraints*** $\Omega$ among the variables ($\emptyset \Rightarrow unconstrained$);
  - An ***objective function*** $f: S \rightarrow \mathbb{R}_0^+$ that is to be minimized.
- From now on we will assume that an instance of a CPO is represented with a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.




# Ant Colony Optimization

**Metaheuristic, adaptation to the problem and resolution.**

# ACO's constructive approach

- ACO metaheuristic's approach is:
  - **Constructive**: a set of solutions is **incrementally** generated at time $t$ taking into account:
    1. Heuristic information about the problem instance being solved, if available (**prior knowledge**);
    2. Pheromone trails which change dynamically at run-time (**acquired search experience**);
  - **Stochastic**: the construction step is chosen in a probabilistic way within a set of components that provide an improvement to the current state of the solutions.

# ACO's approach to COPs

```
procedure ACO algorithm for COPs
  Initialization
  while (not termination cond) do
    ConstructAntSolutions
    ApplyLocalSearch
    GlobalUpdatePheromones
  end
end ACO algorithm for COPs
```

# ACO: Initialization

- Initialization of the **pheromone variables $\tau_0$**
  - Constant — No bias: an edge is chosen randomly
  - Random — Bias is inducted by the randomness
  - Handcrafted — Danger.
- Initialization of the **solution $s_0$**
  - Empty: $s_0 = \emptyset$

# ACO: `ConstructAntSolutions`

- Considering $s_p$ the *partial solution built* at the previous time step and $\mathcal{N}(s_p) \subseteq C$ the *set of feasible components* that can be added to the current partial solution, we choose the next component **stochastically** with the **ant system**:

$$p(c_i^j | s_p) = \frac{\tau_{ij}^{\alpha}(t) \cdot \left[\eta(c_i^j)\right]^{\beta}}{\sum_{c_i^l \in \mathcal{N}(s_p)} \tau_{il}^{\alpha}(t) \cdot [\eta(c_i^l)]^{\beta}}, \qquad \forall c_i^j \in \mathcal{N}(s_p)$$

where:

- $\eta(\cdot)$ is the function that determines the **prior heuristic knowledge** for each feasible solution component $c_i^j$;
- $\tau_{\mathrm{ij}}(\mathrm{t})$ is the **pheromone concentration** for the connection between the current solution and the feasible component $j$ at time $t$;
- $\alpha$ and $\beta$ are **weight parameters** that control the relative importance of the pheromone versus heuristic information.

# ACO: `GlobalUpdatePheromones`

- The following equation regulates the dynamics of the pheromone, which is inteded to make solution components belonging to good solution more desirable:

$$\tau_{ij}(t) = \underbrace{(1-\rho)\tau_{ij}(t-1)}_{\textbf{Pheromone Evaporation}} + \underbrace{\sum_{s \in S_{upd} | c_i^j \in s_p} g(s_p)}_{\textbf{Pheromone Trailing}}$$

where:

- $\rho$ is a constant called **evaporation rate**;
- $\mathrm{S_{upd}}$ is the **set of solutions** that are used to deposit pheromone;
- $g(\cdot)\colon S \to \mathbb{R}^+$ is a function s.t. $f(s) < f(s') \Rightarrow g(s) \geq g(s')$. It determines the quality of a solution and it is called **evaluation function**.

```
procedure ACO_for_TSP(𝒢, m, τ_0, α, β, ρ):
  S = [s_0, s_1, …, s_m] where ∀s_i ⇒ s_i[0] ∈ 𝒢[0] (𝒢[0] is 𝒱);
  τ = τ_0;
  while (not termination cond) do

    ConstructAntSolutions: starting from the initial city s_i[0], the
      i-th ant iteratively adds components to the path from the set
      𝒩(s_i) of the cities that the i-th ant has not visited yet. The
      heuristic information is given by η_ij = 1/d_ij;

    S = ConstructAntSolutions(𝒢, S, τ, α, β);

    GlobalUpdatePheromones: pheromone trails are updated by using
      S_upd = S and g(s_i) = 1/f(s_i) with f length of the tour;

    τ = GlobalUpdatePheromones(S, ρ);
  end
```

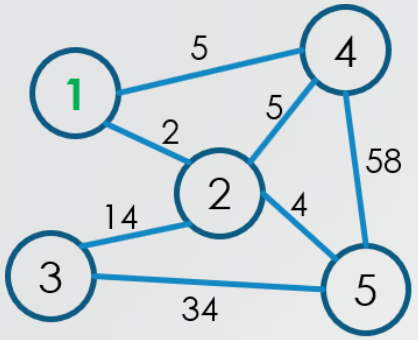
# The first ACO algorithm: Ant System and TSP

# ACO: similarity with ANN

- Some researchers have drawn similarities with the **ANNs** and their way they perform learning [Ahmed et al., 2012] (i.e. the connectionist approach, the numerical embedding of the knowledge, the reinforcement of portions of good solutions);

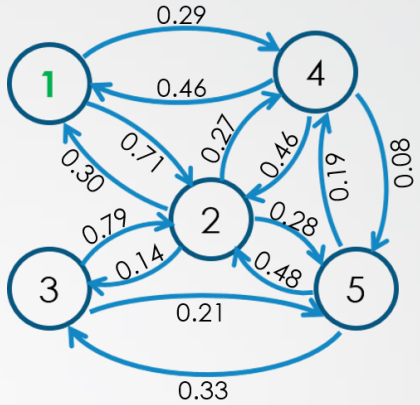
- This similarity has resulted quite forced, because of the confusion made between the concepts of **learning** and **decision process**:
  - ACO is a metaheuristic that **implements an optimization method** → produces a **decision process** for the current COP;
  - ANN is a model that **implements a decision process** as a non-linear function → it has no learning embedded in it.

But ACO produces a decision process which is **constructive** and **stochastic**…




Weight → Cost of the edge

Heuristic modelled as a transition probability




Weight → Transition Probability

- Given this representation of the COP, it is straightforward to understand what follows:
  - **Ants** are modelled as a **sets of random variables**, and the solution provided by the $i$-$th$ ant is given by the **joint state assignment** $\boldsymbol{X_{i0}, X_{i1}, \ldots, X_{it}}$;
  - **Parameterization**: $\mathcal{M} = \{\pi, A, \mathrm{T}\}$ is Markov chain, where
    - $\pi_i$ is the **prior probability** of an ant being in the node $i$ in the state $t = 0$,
    - $A$ is the **prior transition probability** (heuristic information, it already takes in account $\alpha$) and
    - $\mathrm{T}$ is an unconstrained parameter representing **pheromone concentration** (**reparameterized by marginalization or softmax basis** by weighting with $\beta$);
  - **Learning**: is performed as before and must take in account the reparameterization of the matrix $\mathrm{T}$.

# ACO as a Markov chain

# ACO: conclusions

- The approach of ACO is classified as **explicit** because of the ants' stigmergic behaviour, which leads them to the local best solution;
- ACO uses a constructive and stochastic approach to build a solution;
- The learning process is performed by an iterative evaluation of the current solutions and with a consequential improvement of the paths which lead to the best ones.

# Particle Swarm Optimization

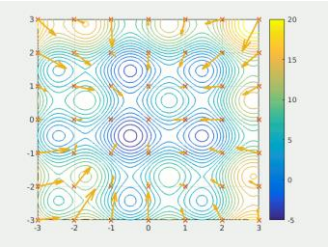
Developed by Kennedy and Eberhart in 1995

Birds flocking behaviour

Modelling

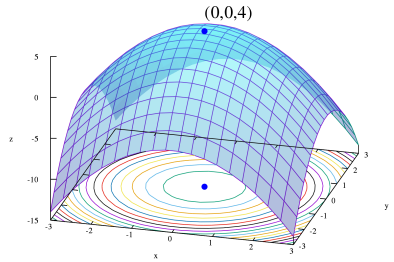
Particles

Behaviour extraction

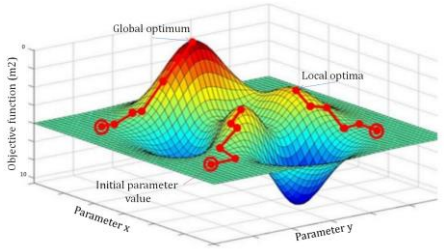


Particle Swarm Optimization

Adaptation to the problem




Continuous Optimization Problem




Local or global optimum

# Birds Flocking Behaviour

**System under study**

# Birds in nature

- Flocking is the main **social behaviour** of the birds;
- Why flocking?
  - To collectively search food;
  - To protect themselves by predators;
- Birds have social interaction which enables them to:
  1. Flying without collisions even with sudden changes of direction;
  2. Scattering and quickly regrouping when reacting to external threats;
  3. Avoiding predators.

# Birds flocking vs. humans' social behaviour

- Society is our version of the flock → we adjust ideas, beliefs and attitudes;
- This adjustments are given by two components:
  - A sort of **common sense** of the society → they are not given only by the local neighbourhood;
  - Our private thinking and reasoning.

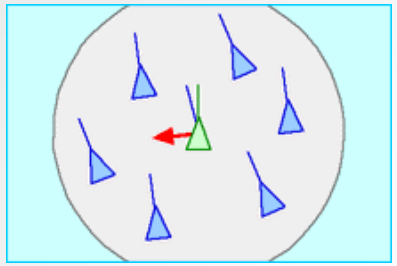
# From Bird Flocks to Particles

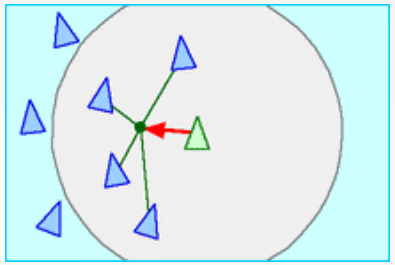
**Artificial Model**

# First flocking simulation: Boids

- It is an artificial life program, developed by Reynolds in 1986;
- The dynamics are composed of three simple rules:
  1. **Alignment**: steer towards the average heading of local flockmates;
  2. **Cohesion**: steer to move towards the average position of local flockmates;
  3. **Separation**: steer to avoid crowding local flockmates.
- The pseudo-randomness of the movement is induced by the **moving center of mass** provided by the flockmates.

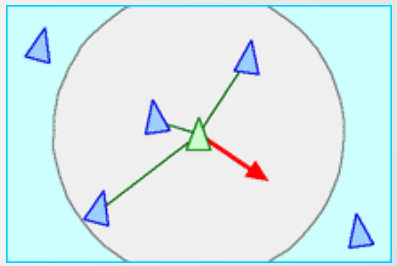
**Alignment**





**Cohesion**

**Separation**



# Caring of the roost: Heppner's simulation

- In this simulation a *roost* is added as **external mass center**;
- The model is given through a **stochastic differential equation driven by a Poisson process**:

$$\begin{cases} dv_i(t) = (F_{home} + F_{vel} + F_{interact})dt + dP(t) \\ v_i(t) = du_i(t), \qquad i = 1, 2, \dots, n \end{cases}$$

where:

- $u_i(t) = (x_i(t), y_i(t))$ is the **location** of the $i$-th bird at time $t$;
- $F^i_{home} = -u_i(t) \cdot f_{home}(|u_i(t)|)$ with $i = 1, 2, \dots, n$ is the **homing factor**, which expresses the tendency of a bird through the roost;
- $F^i_{vel} = -v_i(t) \cdot f_{vel}(|v_i(t)|)$ with $i = 1, 2, \dots, n$ is the **velocity control term**;
- $F^i_{interact} = \sum_{j=1}^{n} f_{interact}\big(|u_j(t) - u_i(t)|\big)(u_j(t) - u_i(t))$ determines **the relative influence** of each bird on its flockmates;
- $P(t)$ is the **random impact term**, modelled by an $n$-dimensional, time-homogeneous Poisson process
- In this simulation the flock converges to the roost;

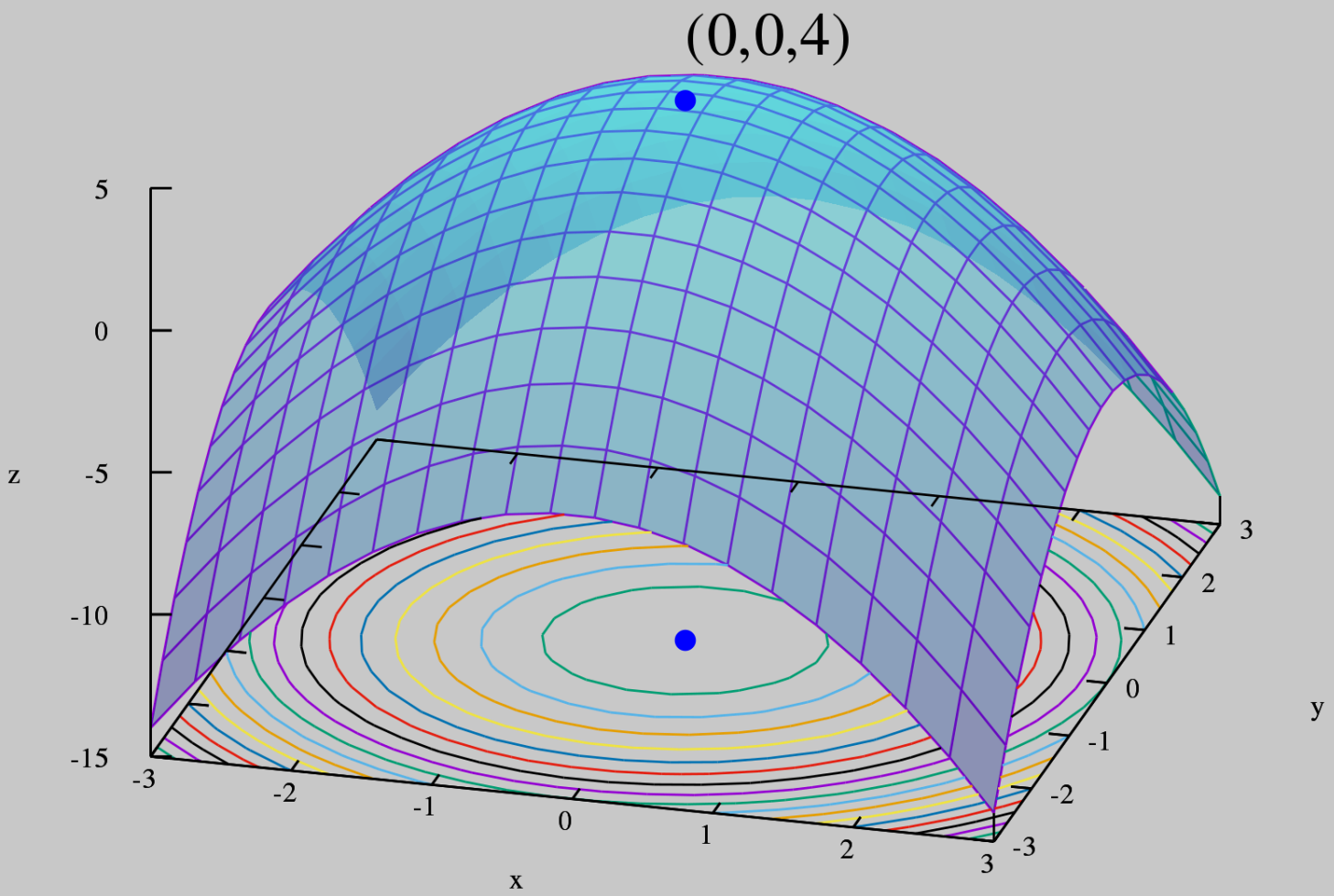
# On the need of food: cornfield vector

- Birds do not land necessarily on the roost, but any spot which satisfies their **immediate needs**;
- Generally these needs are represented by food → introduction of the **cornfield vector**:

$$Eval = \sqrt{(presentx\ - 100)^2} + \sqrt{(presenty\ - 100)^2}$$

  which is the distance from (100, 100), that represents the position of the cornfield vector.

- From now on, our **immediate need is represented by the loss function to be minimized**.




# Continuous Optimization Problems

**Problem Model**

# Continuous Optimization Problems: definition

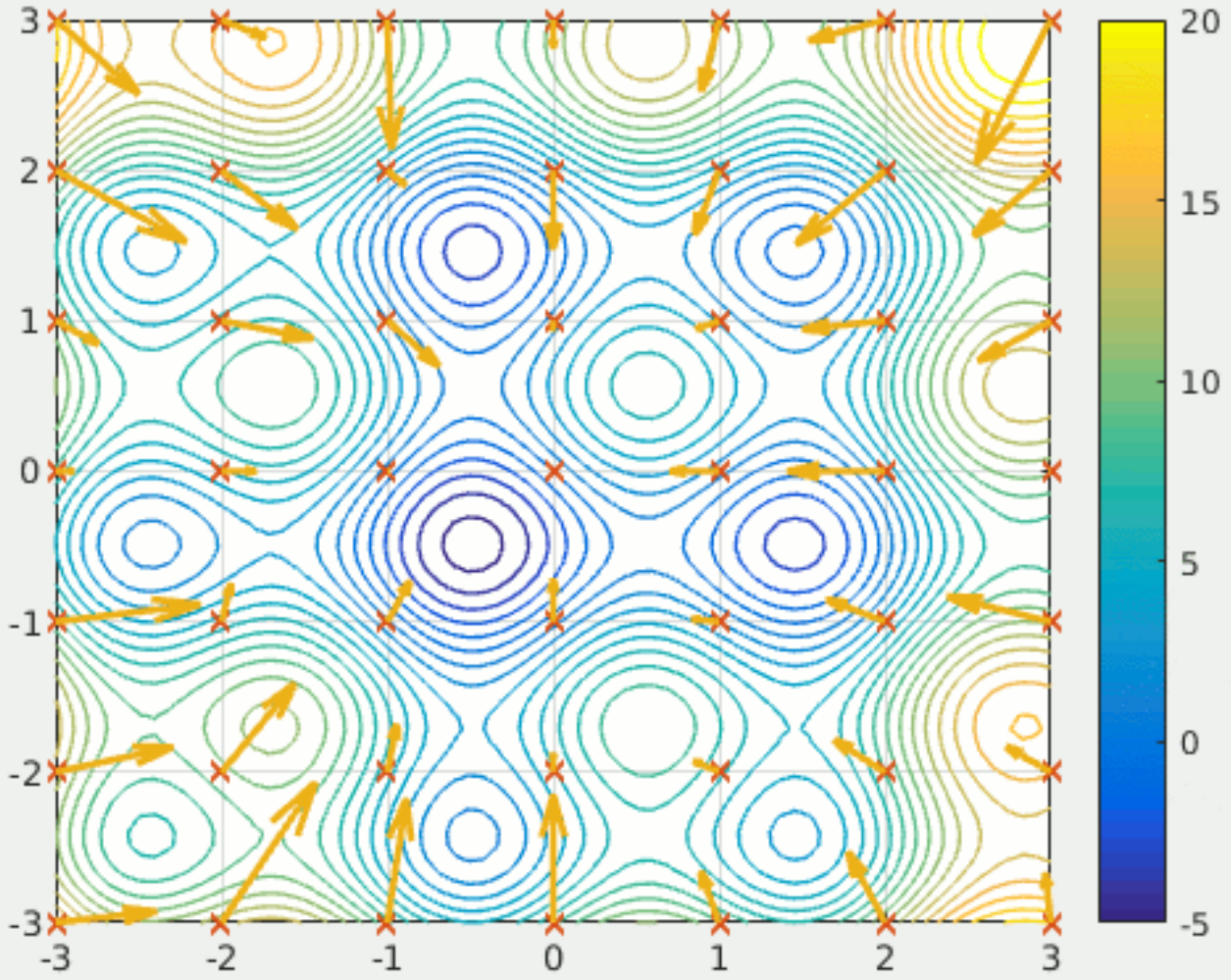
- The standard form of a *continuous optimization problem* is

$$\begin{cases} \min_{\mathrm{x}} f(x) \\ g_i(x) \leq 0, & i = 1, \dots, m \\ h_j(x) = 0, & j = 1, \dots, p \end{cases}$$

where:

- $f: \mathbb{R}^n \to \mathbb{R}$ is the **objective function** to be minimized over the $n$-variable vector $x$;
- $g_i(x) \leq 0$ are called **inequality constraints**;
- $h_j(x) = 0$ are called **equality constraints**;
- $m \geq 0$ and $p \geq 0$ .

- From now on we will deal only with *unconstrained continuous optimization problems* ($m = 0$ and $p = 0$).




# Particle Swarm Optimization

**Metaheuristic**

# PSO: global algorithm

- PSO is an optimization metaheuristic which puts together all the intuitions we have drawn until now:

```
procedure PSO_Global(n_particles):
  particles, p_global = Initialization(n_particles);
  while (not termination cond) do:
    for each v, x, _, p_best in particles:
      v, x = UpdateVelocityAndPosition(v, x, p_best, p_global);
    for each v, x, eval_best, p_best in particles:
      eval_curr = EvaluateParticle(x);
      if eval_curr < eval_best:
        eval_best = eval_curr;
        p_best = x;
        if eval_best <EvaluateParticle(p_global):
          p_global = x;
  return p_global;
```

- `Initialization`: randomly assigning **initial velocity and position** and detects the position of the best particle in the swarm;
- `EvaluateParticle`: **computes the loss** for the position of the current particle;

# PSO: `UpdateVelocityAndPosition`

- The following formula is the **core update** of the algorithm, and it is composed by two simple *velocity* and *position* equations for the $i$-th particle:

$$\boldsymbol{v}_{\mathrm{i}}(\mathrm{t}+1) = \boldsymbol{v}_{\mathrm{i}}(\mathrm{t}) + \mathrm{c}_1\mathbf{R}_1\left(\mathbf{p}_{\mathrm{ibest}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t})\right) + \mathrm{c}_2\mathbf{R}_2(\mathbf{p}_{\mathrm{global}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t}))$$
$$\mathbf{x}_{\mathrm{i}}(\mathrm{t}+1) = \mathbf{x}_{\mathrm{i}}(\mathrm{t}) + \boldsymbol{v}_{\mathrm{i}}(\mathrm{t}+1)$$

where:

- $\boldsymbol{v}_i$ represents the **rate of position change** along all the dimensions;
- $\mathbf{x}_{\mathrm{i}}$ represents the **position** of the particle;
- $\mathbf{p}_{\mathrm{ibest}}$ represents the **historically best position** of the particle;
- $\mathbf{p}_{\mathrm{global}}$ represents the **position of the swarm's best particle**;
- $\mathbf{R_1}, \mathbf{R_2} \in [\mathbf{0}, \mathbf{1}]^{\mathbf{d}}$ introduce randomness in the search strategy;
- $\mathrm{c}_1, \mathrm{c}_2$ are positive constant parameters.

# PSO: velocity equation interpretation

$$\boldsymbol{v}_{\mathrm{i}}(\mathrm{t}+1) = \underbrace{\boldsymbol{v}_{\mathrm{i}}(\mathrm{t})} + \underbrace{\mathrm{c}_1\mathbf{R}_1\left(\mathbf{p}_{\mathrm{ibest}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t})\right)} + \underbrace{\mathrm{c}_2\mathbf{R}_2\left(\mathbf{p}_{\mathrm{global}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t})\right)}$$

**Momentum**: ensures that the velocity of a particle does not change abruptly

**Cognitive part**: implements the linear attraction towards the historically best position of the particle.

**Social part**: implements a linear attraction towards the globally best position ever found by any particle

# PSO refinements: limiting the maximum velocity

- **Issue**: when $\mathbf{x}_\mathrm{i} \ll \mathbf{p}_\mathrm{ibest}$ and $\mathbf{x}_\mathrm{i} \ll \mathbf{p}_\mathrm{global}$ this caused swarm explosion (i.e. uncontrolled increase of the magnitude $|\boldsymbol{v}_i(t+1)|$));

- **Solution**: thresholding of the velocity vector with $v_{max}$

$$v_{id}(t+1) = \begin{cases} v_{max}, & if\ v_{id}(t+1) > v_{max} \\ -v_{max}, & if\ v_{id}(t+1) < -v_{max} \end{cases} \qquad \text{with } v_{max} = \frac{x_d^{max} - x_d^{min}}{K}$$

- This definition of $v_{max}$ with the user-defined parameter $K$ allows a shift of a particle which is linear w.r.t. the magnitude of the dimension

# PSO refinements: Inertia weight

- **Issue**: premature convergence through local optima due to the massive weight of the momentum;

- **Solution**: introduction of a weighting of the momentum via inertia

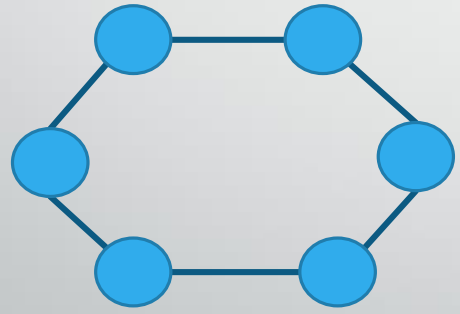
$$\boldsymbol{v}_{\mathrm{i}}(\mathrm{t}+1) = \boldsymbol{\omega}(\mathrm{t})\boldsymbol{v}_{\mathrm{i}}(\mathrm{t}) + \mathrm{c}_1\mathbf{R}_1\left(\mathbf{p}_{\mathrm{ibest}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t})\right) + \mathrm{c}_2\mathbf{R}_2(\mathbf{p}_{\mathrm{global}}(\mathrm{t}) - \mathbf{x}_{\mathrm{i}}(\mathrm{t}))$$

$$\boldsymbol{\omega}(\mathrm{t}) = \omega_{\mathrm{hi}} - (\omega_{\mathrm{hi}} - \omega_{\mathrm{low}}) \cdot \frac{\mathrm{t}}{\mathrm{t}_{\mathrm{max}}}$$

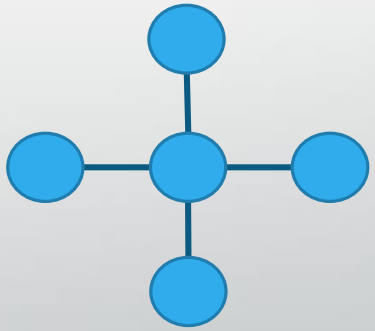
- A high inertia value in the initial iterations allow a fast motion through an optimum, and a lower value in the latter ones allows a wider exploration of the single particles.

# PSO refinements: neighbourhood topology

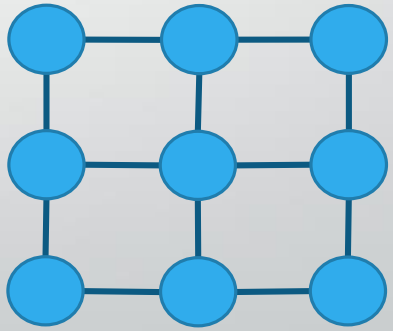
- **Issue**: the instant passing of the information about the swarm's best position causes premature convergence
- **Solution**: the information about the swarm optimum computed and propagated only in terms of a defined local neighbourhood;
- This allows a slower propagation of the information → more freedom for the particles for their search in the space;
- **Examples** of topologies:



**Circle**



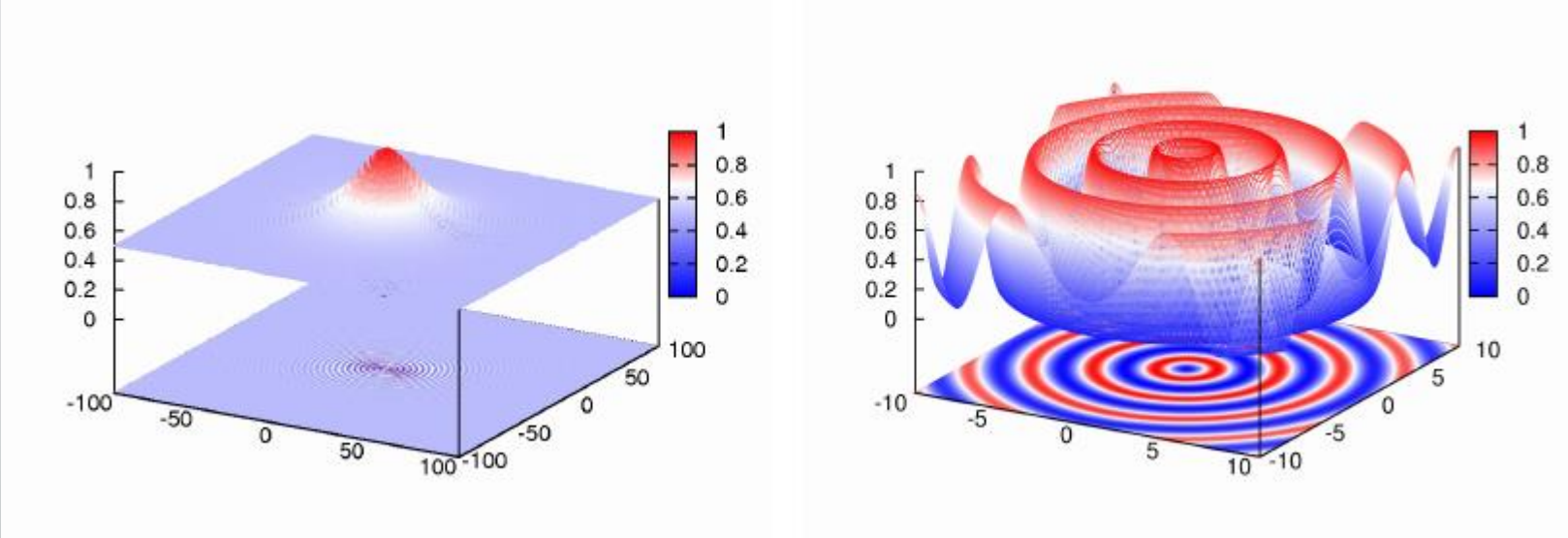
**Star**



**Von Neumann Topology**

# PSO: examples of applications

- PSO has produced optimal results for **highly non-linear functions** such as Schaffer's F6 function:




The algorithm has reached convergence to the global optimum in all the tests

- PSO has shown to be capable of producing really good results in the training of MLPs even without any kind of first order information.

# PSO: discussion

- It performs an **implicit search** of the optimal solution, relying on the swarm's collective knowledge and its dynamics;

- It has two main **points of strength**:
  - Its **simplicity** allows it to be implemented in really few minutes in an efficient way;
  - It is **adaptable to dynamical environments**;

- However, it meets **some issues**:
  - It cannot be used in problems whose solutions are not mapped in an $n$-dimensional Cartesian space;
  - The assumption about homogeneous knowledge of all the particles ($c_1$ and $c_2$ constant) can be restrictive;
  - It fails to locate many local optima;

# Swarm Intelligence: conclusions

- We proposed a general framework to approach optimization problems with the swarm intelligence techniques;
- The two main techniques of this field have been treated in their basic form (many extensions exist in literature);
- A personal consideration: my hope that the direction will be the one to concile the existing well known approaches with the swarm intelligence.

# Thank you for your attention!

# References


- Dorigo, Marco, Vittorio Maniezzo, and Alberto Colorni. "**Ant system: optimization by a colony of cooperating agents**." *IEEE Transactions on Systems, man, and cybernetics, Part B: Cybernetics* 26.1 (1996): 29-41.
- Dorigo, Marco, and Luca Maria Gambardella. "**Ant colony system: a cooperative learning approach to the traveling salesman problem**." *IEEE Transactions on evolutionary computation* 1.1 (1997): 53-66.
- Reynolds, Craig W. "***Flocks, herds and schools: A distributed behavioral model.***" Vol. 21. No. 4. ACM, 1987.
- Heppner, Frank, and Ulf Grenander. "**A stochastic nonlinear model for coordinated bird flocks.**" *The ubiquity of chaos* 233 (1990): 238.
- J. Kennedy and R. Eberhart, "**Particle swarm optimization.**" *Proceedings of ICNN'95 - International Conference on Neural Networks*, Perth, WA, Australia, 1995, pp. 1942-1948 vol.4.
  doi: 10.1109/ICNN.1995.488968
- Eberhart, Russell, and James Kennedy. "**A new optimizer using particle swarm theory**." *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science*. Ieee, 1995.
- Ahmed, Hazem, and Janice Glasgow. "**Swarm intelligence: concepts, models and applications**." *School Of Computing, Queens University Technical Report* (2012).